SecEQP: A Secure and Efficient Scheme for SkNN Query Problem over Encrypted Geodata on Cloud

Alex X. Liu

Professor, IEEE Fellow

Dept. of Computer Science & Engineering

Michigan State University

East Lansing, Michigan

Co-authors: Xinyu Lei, Rui Li, and Guan-Hua Tu




c|net

Privacy Matters

- **Facebook–Cambridge Analytica data scandal in 2018**

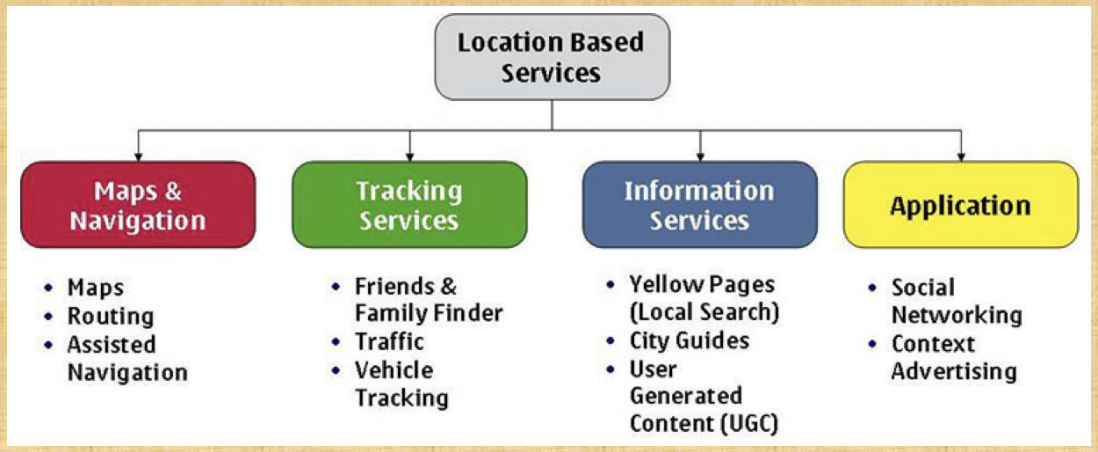


- **"Outsourced data storage on remote clouds is practical and relatively safe if only the data owner, not the cloud service, holds the decryption keys."**
  - **The General Data Protection Regulation (GDPR) is a regulation in EU law on data protection and privacy for all individuals within the European Union (EU) and the European Economic Area (EEA).**
  - **In effective since May 2018.**

Location Based Services vs. Location Privacy

- Location Based Services





- Location Privacy







Why Cloud Cannot Be Fully Trusted

- Cloud may give your personal data to government or another company



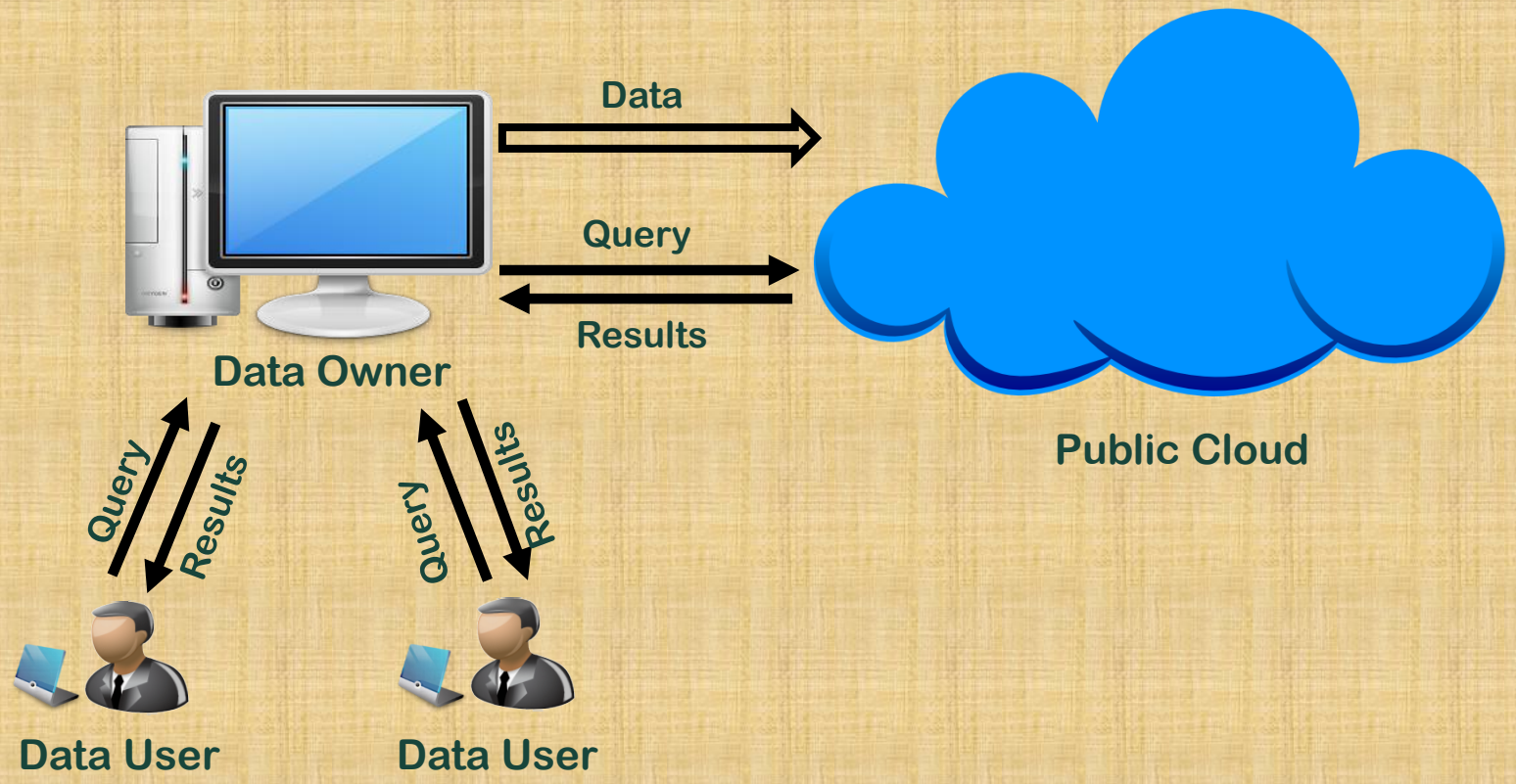
- Corrupted cloud employee may peak at your data



- Cloud may be hacked



System and Threat Model



- Threat Model: semi-honest (i.e., honest-but-curious)

Problem Statement

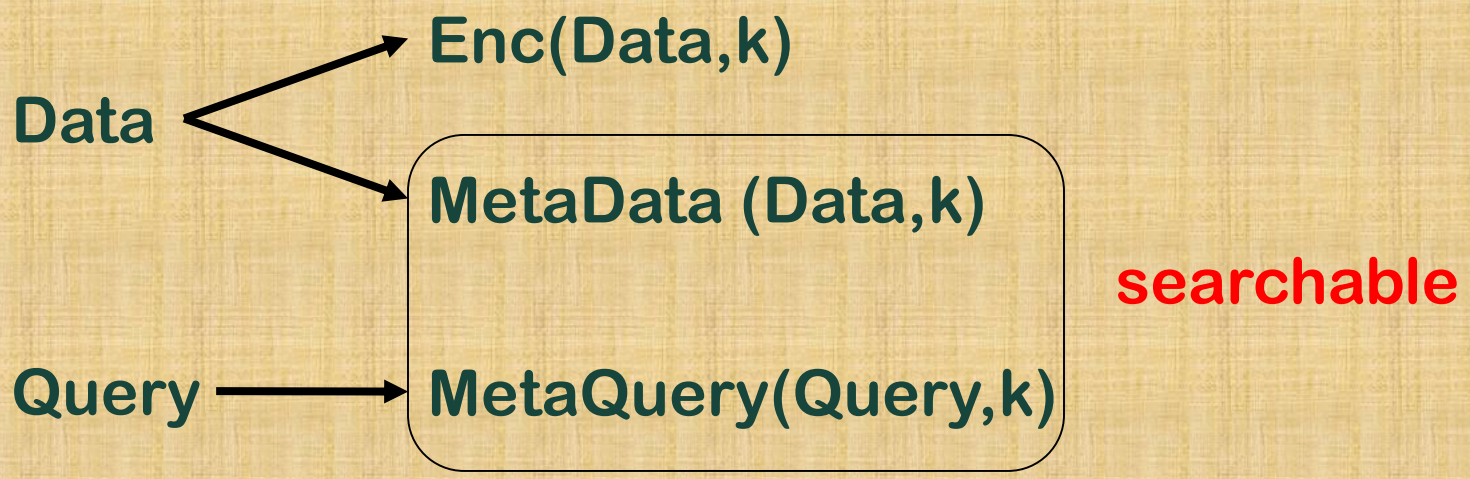
- Problem: Searchable Symmetric Encryption for kNN Geolocation Queries

- Data: 2-D geospatial data
- Query: kNN query for a given location

- Requirement
  - Security: provable
  - Practicality: usable
    - Efficiency: ms for querying millions of data points
    - Scalability: sub-linear

Search over Encrypted Data

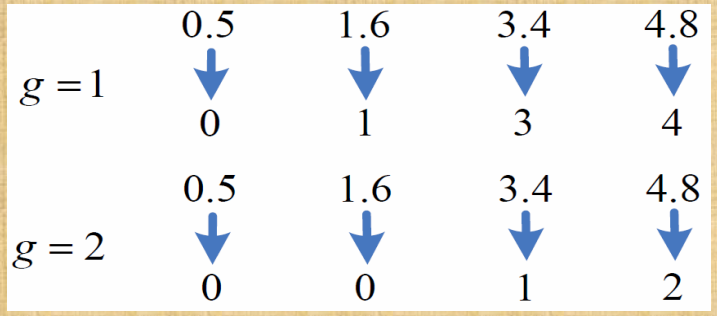
- Encrypted data themselves are not searchable.



- MetaData is called Secure Index
- MetaQuery is called Trapdoor

kNN Query Processing in 1-D Data

- Distance between 1-D data points p and q = |p-q|.
  - If p and q are plain text: trivial.
  - If p and q are encrypted: requires homomorphic encryption (extremely slow)
- How to avoid computation over encrypted data?
  - Idea 1: segmentation with controllable granularity
    - For each data point p, convert p to $\lfloor \frac{p}{g} \rfloor$, where g is the granularity.



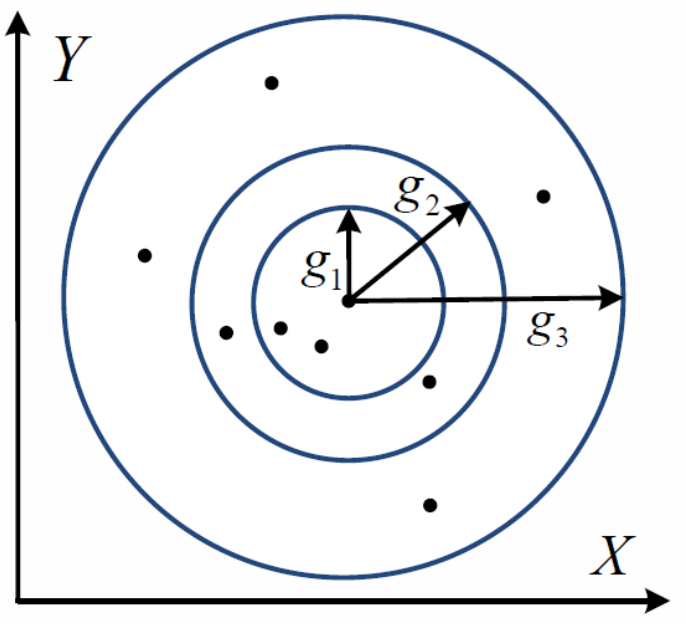
  - Idea 2: checking whether two numbers are equal is easy to do in a privacy-preserving fashion using secure hash functions
    - Given data p_1, …, p_n , compute HMAC($\lfloor \frac{p_1}{g} \rfloor$,k), …, HMAC($\lfloor \frac{p_n}{g} \rfloor$,k).
    - Given query q, computer HMAC($\lfloor \frac{q}{g} \rfloor$,k).

kNN Query Processing in 2-D Data

- Distance between two 2-D points (x_1, y_1) and (x_2, y_2)

$$= \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

- Granularity is in terms of circles, not segments.



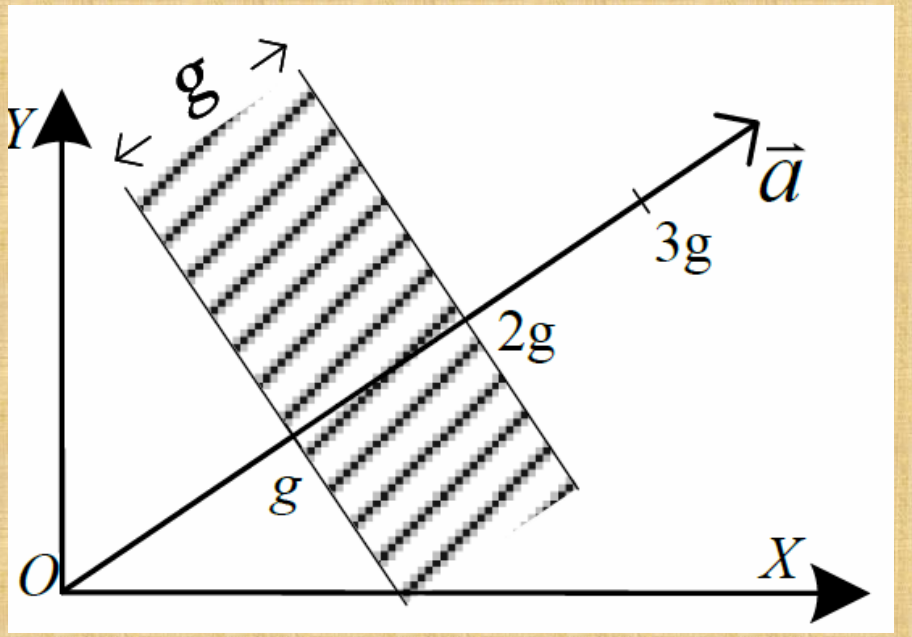
- How to check whether two points are in the same circle?
  - Idea 3: Multi-vector Based Segmented Projection

Multi-vector Based Segmented Projection

- 1-Vector based segmented projection:
  - Given data p, segment length g, and a unit vector $\vec{a}$, (i.e.,$|\vec{a}|$=1),

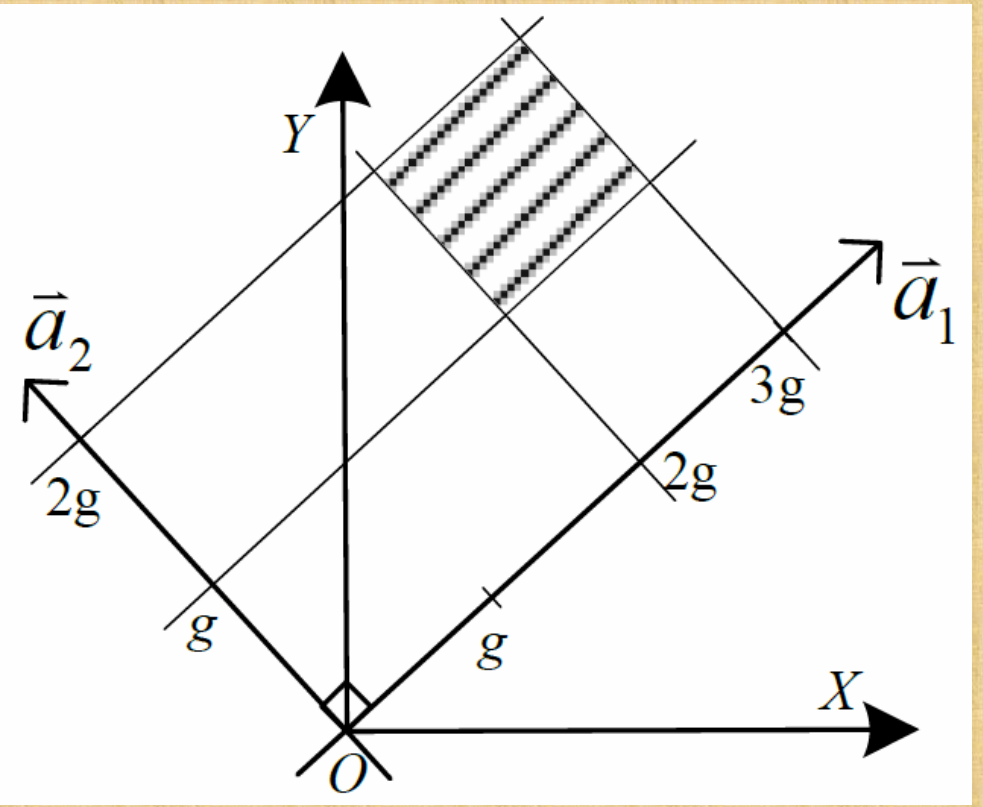
$$h_{\vec{a},g}(p) = \lfloor \frac{\vec{p}.\vec{a}}{g} \rfloor$$

  - Equivalence: $p_1 \equiv p_2$ iff $h_{\vec{a},g}(p_1) = h_{\vec{a},g}(p_2)$.
  - Geometrically, equivalent class is a bar.



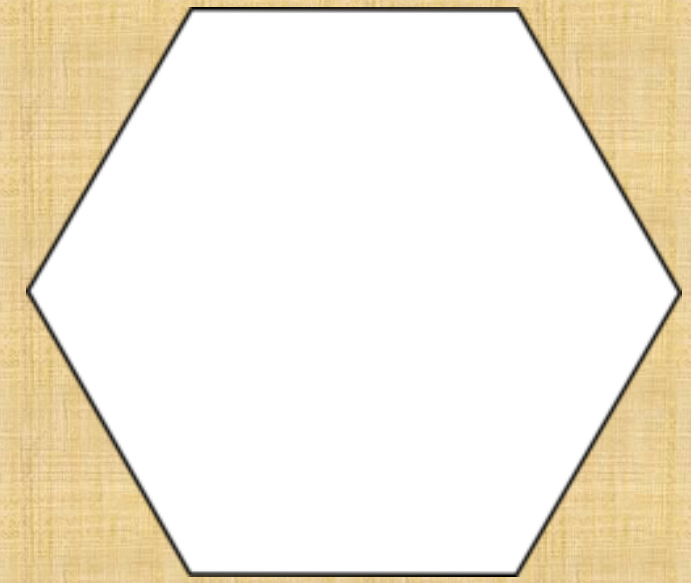
Multi-vector Based Segmented Projection

- 2-Vector based segmented projection:
  - Degree between two vectors: 360/(2*2)=90
  - Equivalence: $p_1 \equiv p_2$ iff $h_{\overrightarrow{a_1},g}(p_1) = h_{\overrightarrow{a_1},g}(p_2)$ and $h_{\overrightarrow{a_2},g}(p_1) = h_{\overrightarrow{a_2},g}(p_2)$
  - Geometrically, equivalent class is a square.



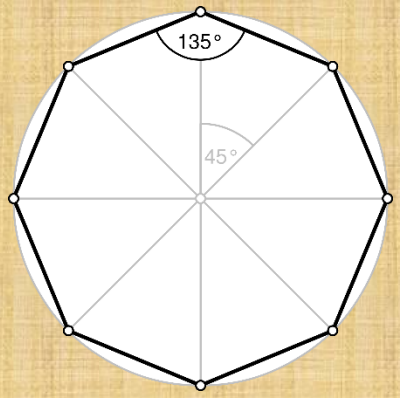
Multi-vector Based Segmented Projection

- 3-Vector based segmented projection:
  - Degree between two vectors: 360/(2*3)=60
  - Equivalence: $p_1 \equiv p_2$ iff
    - $h_{\overrightarrow{a_1},g}(p_1) = h_{\overrightarrow{a_1},g}(p_2)$ and
    - $h_{\overrightarrow{a_2},g}(p_1) = h_{\overrightarrow{a_2},g}(p_2)$ and
    - $h_{\overrightarrow{a_3},g}(p_1) = h_{\overrightarrow{a_3},g}(p_2)$
  - Geometrically, equivalent class is a regular hexagon.

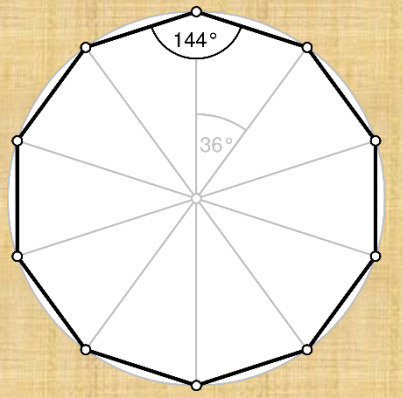


Multi-vector Based Segmented Projection

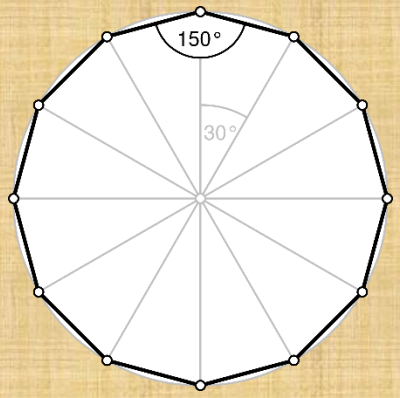
- **d-Vector based segmented projection:**
  - Degree between two vectors: 360/(2*d)
  - Equivalence: $p_1 \equiv p_2$ iff
    - $h_{\overrightarrow{a_1},g}(p_1) = h_{\overrightarrow{a_1},g}(p_2)$ and
    - $h_{\overrightarrow{a_2},g}(p_1) = h_{\overrightarrow{a_2},g}(p_2)$ and
    - .....
    - $h_{\overrightarrow{a_d},g}(p_1) = h_{\overrightarrow{a_d},g}(p_2)$
  - Geometrically, equivalent class is a regular polygon with 2d edges.
  - The larger d is, the more closer the equivalent class is a circle.



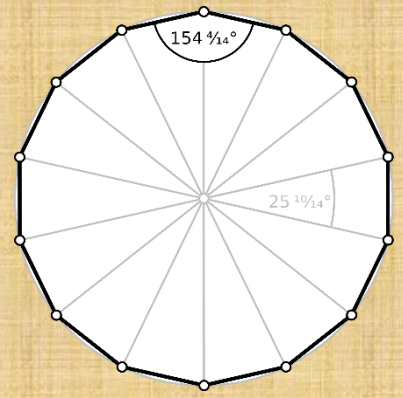
d=4



d=5



d=6



d=7

Data Processing with d Vectors and m Granularities

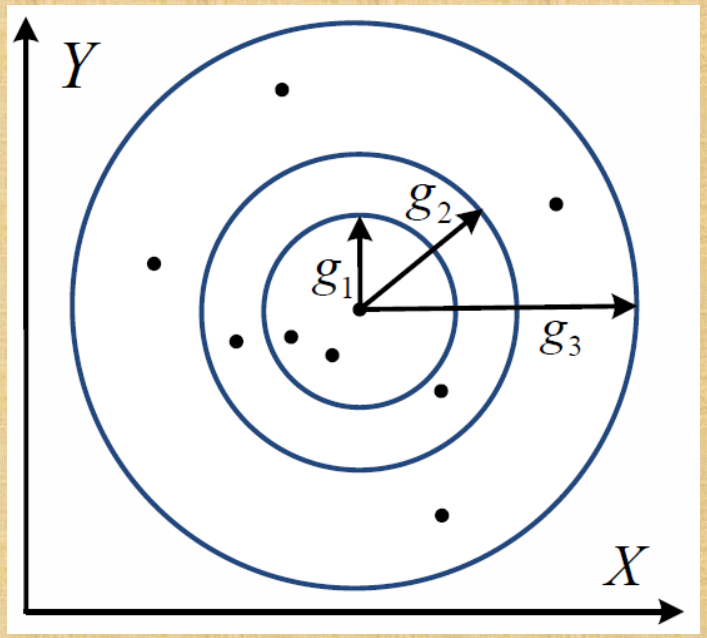
- For each data point p_i:
  - for granularity g_1, compute: $h_{\overrightarrow{a_1},g_1}(p_i)$, $h_{\overrightarrow{a_2},g_1}(p_i)$,…, $h_{\overrightarrow{a_d},g_1}(p_i)$
  - for granularity g_2, compute: $h_{\overrightarrow{a_1},g_2}(p_i)$, $h_{\overrightarrow{a_2},g_2}(p_i)$,…, $h_{\overrightarrow{a_d},g_2}(p_i)$
  - ……
  - for granularity g_m, compute: $h_{\overrightarrow{a_1},gm}(p_i)$, $h_{\overrightarrow{a_2},gm}(p_i)$,…, $h_{\overrightarrow{a_d},gm}(p_i)$

Basic Linear KNN Query Processing Algorithm

- Linear Algorithm for finding k-nearest neighbors for query q:
  - result = $\emptyset$;
  - for j=1 to m
    - for each data point p_i, if

      $(h_{\overrightarrow{a_1},g_j}(q) = h_{\overrightarrow{a_1},g_j}(p_i)) \wedge (h_{\overrightarrow{a_2},g_j}(q) = h_{\overrightarrow{a_2},g_j}(p_i)) \wedge \ldots \wedge (h_{\overrightarrow{a_d},g_j}(q) = h_{\overrightarrow{a_d},g_j}(p_i))$

      then add p_i to result.
    - if |result|≥k, then exit.



Convert Equality Comparison to Membership Query

- **Convert d equality comparisons to one equality comparison:**

$(h_{\overrightarrow{a_1},g_j}(q) = h_{\overrightarrow{a_1},g_j}(p_i)) \wedge (h_{\overrightarrow{a_2},g_j}(q) = h_{\overrightarrow{a_2},g_j}(p_i)) \wedge \ldots \wedge (h_{\overrightarrow{a_d},g_j}(q) = h_{\overrightarrow{a_d},g_j}(p_i))$

$\rightarrow h_{\overrightarrow{a_1},g_j}(q) | h_{\overrightarrow{a_2},g_j}(q) | \ldots | h_{\overrightarrow{a_d},g_j}(q) = h_{\overrightarrow{a_1},g_j}(p_i) | h_{\overrightarrow{a_2},g_j}(p_i) | \ldots | h_{\overrightarrow{a_d},g_j}(p_i)$

$\rightarrow$ HMAC$(h_{\overrightarrow{a_1},g_j}(q) | h_{\overrightarrow{a_2},g_j}(q) | \ldots | h_{\overrightarrow{a_d},g_j}(q), K)$ = HMAC$(h_{\overrightarrow{a_1},g_j}(p_i) | h_{\overrightarrow{a_2},g_j}(p_i) | \ldots | h_{\overrightarrow{a_d},g_j}(p_i)', K)$

- **Further convert one comparison to membership queries**

HMAC$(h_{\overrightarrow{a_1},g_j}(q) | h_{\overrightarrow{a_2},g_j}(q) | \ldots | h_{\overrightarrow{a_d},g_j}(q), K)$ = HMAC$(h_{\overrightarrow{a_1},g_j}(p_i) | h_{\overrightarrow{a_2},g_j}(p_i) | \ldots | h_{\overrightarrow{a_d},g_j}(p_i)', K)$

$\rightarrow$ Is HMAC$(j | h_{\overrightarrow{a_1},g_j}(q) | h_{\overrightarrow{a_2},g_j}(q) | \ldots | h_{\overrightarrow{a_d},g_j}(q), K)$ in the set

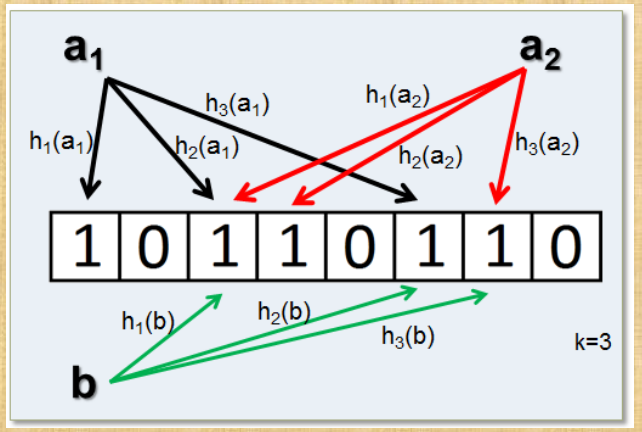
{HMAC$(1 | h_{\overrightarrow{a_1},g_1}(p_i) | h_{\overrightarrow{a_2},g_1}(p_i) | \ldots | h_{\overrightarrow{a_d},g_1}(p_i)', K)$,
HMAC$(2 | h_{\overrightarrow{a_1},g_2}(p_i) | h_{\overrightarrow{a_2},g_2}(p_i) | \ldots | h_{\overrightarrow{a_d},g_2}(p_i)', K)$,
……
HMAC$(m | h_{\overrightarrow{a_1},g_m}(p_i) | h_{\overrightarrow{a_1},g_m}(p_i) | \ldots | h_{\overrightarrow{a_d},g_m}(p_i)', K)$}

- **For each data point, use an Indistinguishable Bloom Filter (IBF) to store its m HMAC values.**

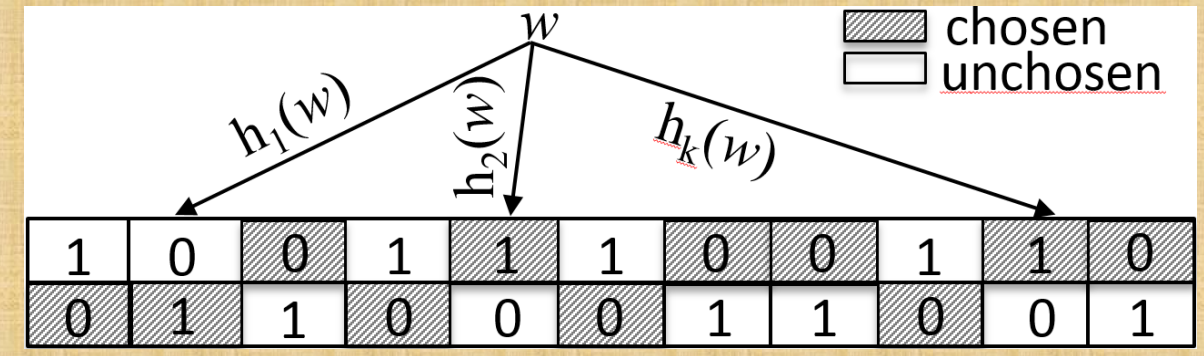
- **Construct a structurally indistinguishable tree from n IBFs.**

Indistinguishable Bloom Filter (IBF)

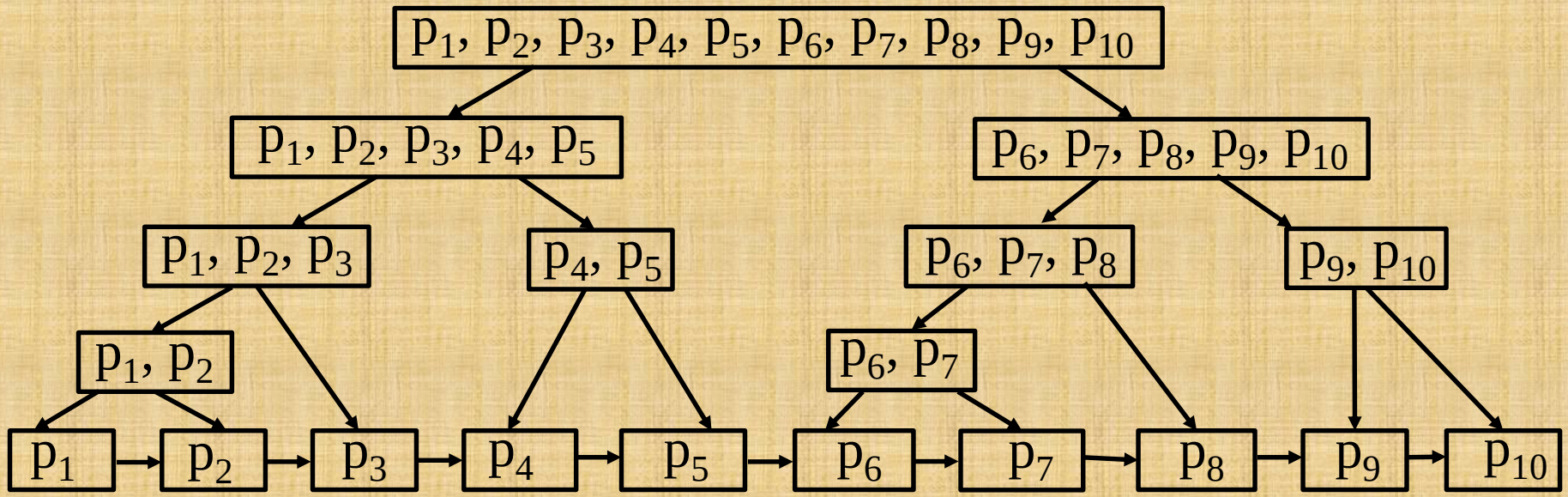
- Bloom Filter:



- Indistinguishable Bloom Filter (IBF)
  - Twin cell: 0 and 1, or 1 and 0
  - For any element e into an IBF, hash r times into BF using r secret keys $k_1,\ldots, k_r$: HMAC(k_1, e), …, HMAC(k_r, e)
  - For the i-th location, which cell stores 1 is determined by another secret key K_{k+1} and a random number for IBF B.
    - The other cell stores 0.



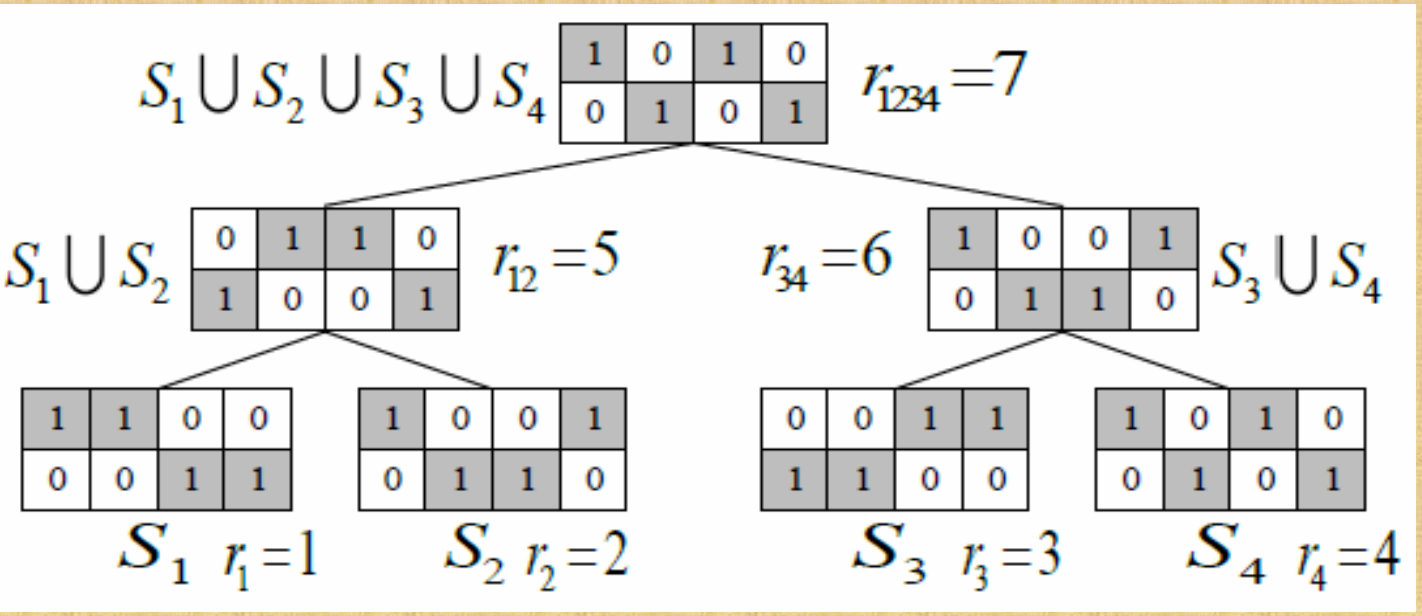
IBTree – Structual Indistinguishability



- Binary
- 0≤|left| - |right|≤1
- Each node is an IBF
- All IBFs in an IBTree have the same length
- Leaves are chained
- Construction is bottom up by logical OR

IBTree Constructed Bottom Up

- IBTree construction is bottom up by logical OR



Security Model

- **Adaptive IND-CKA: indistinguishability against chosen keyword attack**
  - Cloud chooses two distinct sets D_0 and D_1, and sends to data owner.
    - D_0 and D_1 contain equal number of records.
  - Data owner randomly chooses D_0 or D_1
    - Builds metadata I_b for the chosen D_b,
    - Sends I_b to cloud.
  - Repeats the following steps for a polynomial number of times
    - Cloud chooses a query q, sends the query to data owner
      - The query has the same # of satisfying elements in D_0 and D_1.
    - Data owner generates trapdoor t_q and sends t_q to cloud.
    - Cloud uses t_q to query I_b, then chooses a new query based on all pervious queries and query results
  - In the end, cloud guesses b=0/1 still with 50% probability.

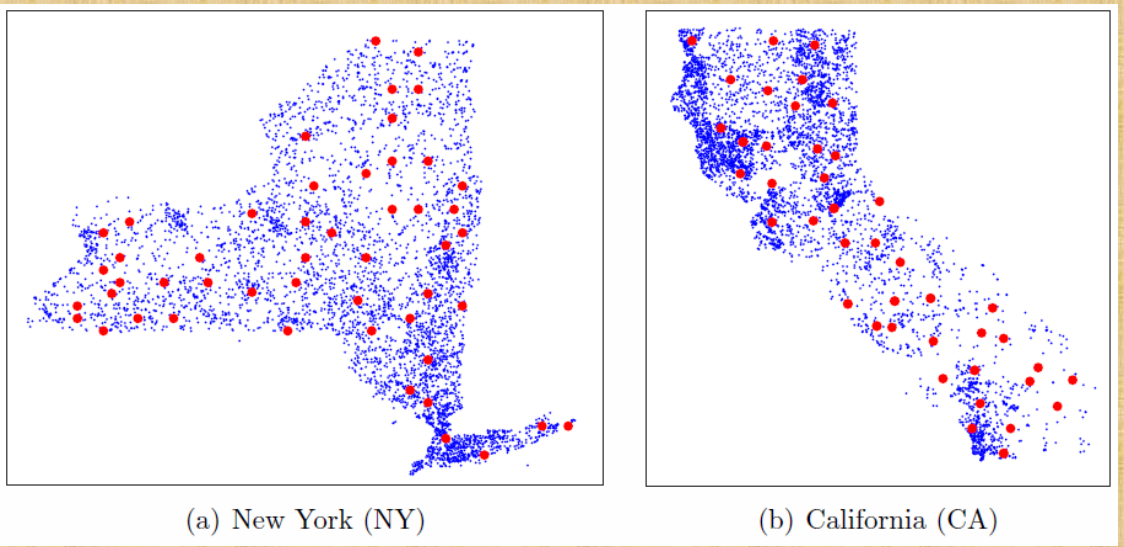
- **We do not hide query patterns and access patterns.**
  - Privacy is already expensive. We do not want absolute privacy.

Proof Sketch

- Leakage function
  - $L_1(I,D)$: given index I and dataset D, outputs
    - Size of each IBF
    - # of data items
    - Data item IDs
    - Size of each encrypted data item
  - $L_2(I,D,q,t)$:given index I,dataset D,atomic query q, time t, outputs
    - Search pattern: whether same search has been performed before
    - Access pattern: which data items satisfies q at time t.
- Simulation based proof: A scheme is adaptively secure iff given index I, we can construct simulated index I' using truly random functions (with an oracle) so that cloud cannot distinguish them for any queries (both known and unknown).
  - To cloud: I = I' with oracle

Experimental Setup

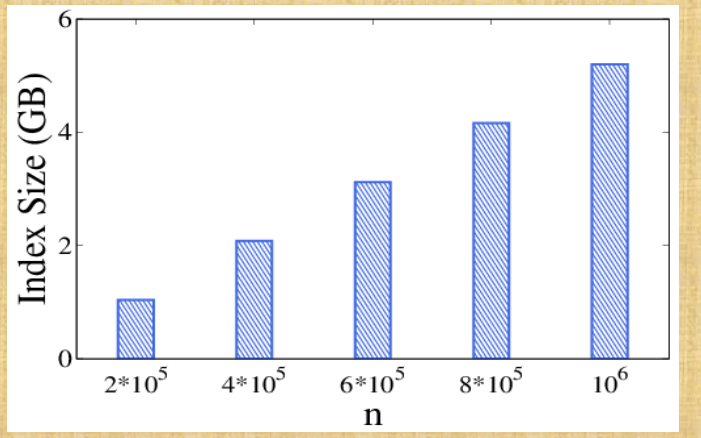
- Data sets:
  - Two real-world data sets
    - 1 million spatial data in the state of New York
    - 1 million spatial data in the state of California



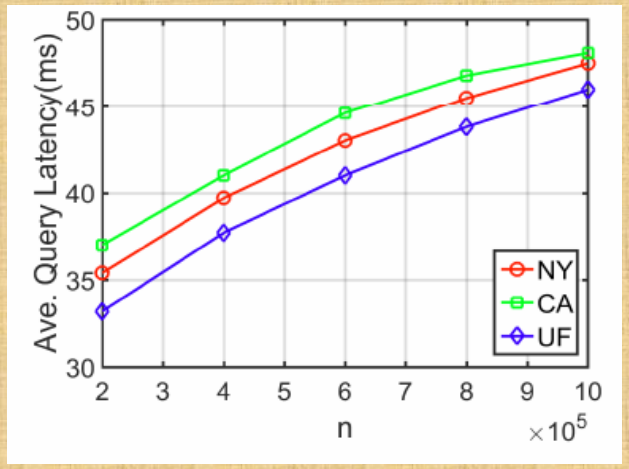
(a) New York (NY) (b) California (CA)

  - One synthetic data set
    - 1 million spatial data generated from uniform (UF) distribution
- Implementation language: C++.
- Machine: 128 GB RAM and two 2.5Ghz 10-core Intel Xeon CPU

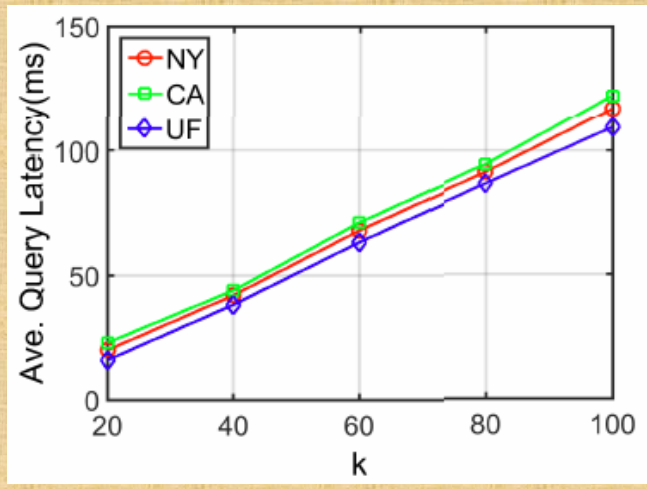
Index size and Query latency



Index size by varying the number of data items n



Query latency by varying number of data items n

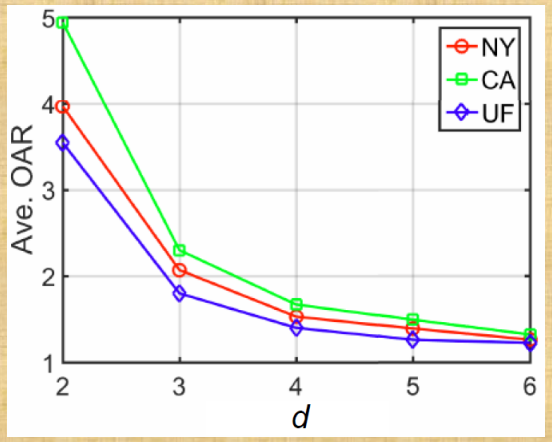


Query latency by varying k

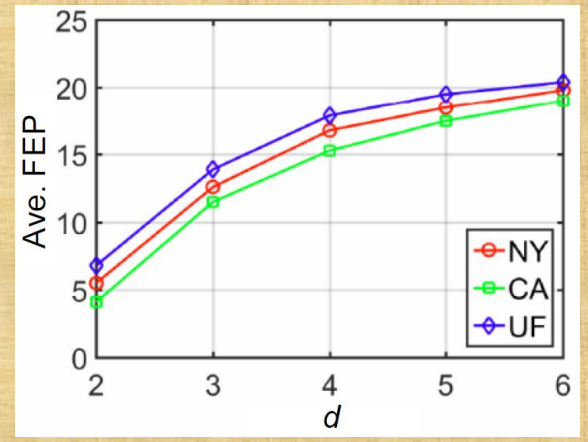
Query Result Accuracy

- Overall Approximation Ratio (OAR) = $\frac{1}{k}\sum_{i=1}^{k}\frac{\|o_i,q\|}{\|o_i^*,q\|}$, where q is the query point, o_i is the ith nearest point in the search result and o_i^* is the actual ith nearest point in the dataset.
- First Error Place (FEP) is the defined as the smallest subscript j such that $\forall i \leq j-1, \|o_i,q\| = \|o_i^*,q\|$ but $\|o_j,q\| \neq \|o_j^*,q\|$.
- Missing Rate = |G-R|/|G|, where G is the set of points from the ground truth and R is the set of points from the search results.
- Redundancy Rate=|R-G|/|R|.

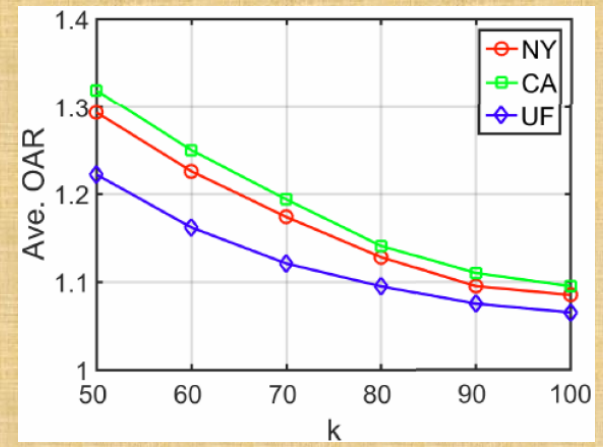
Query Result Accuracy



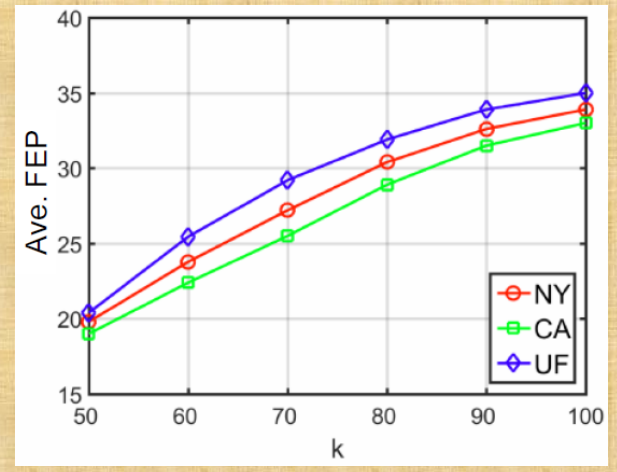
OAR by varying dimension d (50NN)



FEP by varying dimension d (50NN)

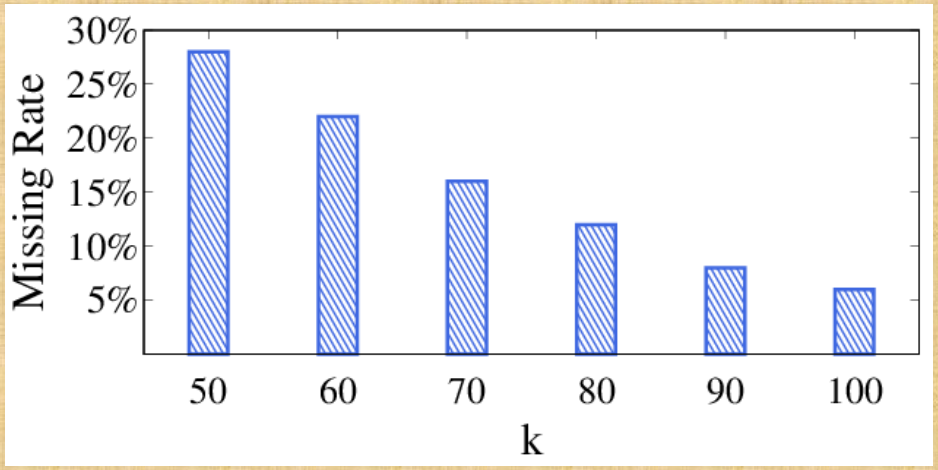


OAR by requiring a larger k and figure out top 50 nearest points for 50NN

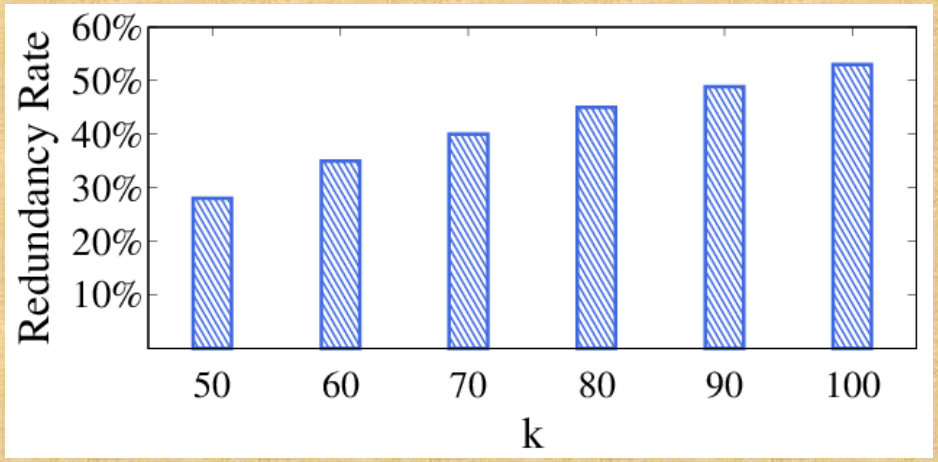


FEP by requiring a larger k and figure out top 50 nearest points for 50NN

Query Result Accuracy



Missing rate by requiring a larger k and figure out top 50 nearest points for 50NN



Redundancy rate by requiring a larger k and figure out top 50 nearest points for 50NN

Questions?

